

Valmiki LMS System

Implementation Plan & One-Week Delivery Schedule

Version	1.1
Date	27 July 2026
Delivery window	Mon 27 Jul – Sun 2 Aug 2026
Roles	Admin · Teacher · Parent
Scope	16 locked flows
Stack	React 19 · Express · Prisma · Supabase

Contents

- 1. Implementation Plan
- 2. One-Week Delivery Schedule
- 3. Deferred — AI Integration (Phase 2)

AI integration deferred to Phase 2

Valmiki LMS System — Implementation Plan

V1.1

Status: Approved, in delivery · Date: 2026-07-27 Sources of truth (priority order): [VIG Developer Document](#) > [Teacher Parent Portal Features](#) > [Product Design System Brand Guidelines](#)

Decisions log (v1.1)

#	Decision	Effect
D1	Product name is Valmiki LMS System . VIG = Valmiki International Gurukulam (the institution). The boards' "LittlePath" wordmark is superseded.	All UI copy, page titles, PWA manifest, PDF header, login screen
D2	Forgot password → admin-mediated . User taps "Forgot password", the system raises a notification to admins, admin resets, admin shares the new temporary password out-of-band. No email/SMS verification.	New PASSWORD_RESET_REQUEST notification + 2 endpoints (\$1.1 F21)
D3	PWA + service worker + VAPID built now ; actual web-push <i>delivery</i> stays behind FEATURE_WEB_PUSH and is the flagged follow-up. In-app notification centre is the launch channel.	M13
D4	All AI is deferred to Phase 2 . Every AI touchpoint ships as a working manual path with the integration seam already in place. See DEFERRED-AI.md .	\$8, M7, M11

AI status: DEFERRED. No AI is wired in this build. Every place AI will eventually plug in is listed in [DEFERRED-AI.md](#) and marked in code with `// TODO(AI-PHASE-2)`. The manual equivalent works end-to-end in the meantime, so nothing in the product is blocked on it.

o. Executive summary

Valmiki LMS System is a homeschool learning portal for **Valmiki International Gurukulam (VIG)**. Three roles (Admin, Teacher, Parent), 16 locked flows, <10 teachers. AI deferred to Phase 2.

The product is one information cycle:

Admin builds structure → Admin schedules → Teacher teaches → Teacher records (voice)
→ AI drafts → Teacher approves → Student record updates → Parent sees approved story

Two architectural invariants drive everything:

- The Class Occurrence is the single anchor.** Attendance, class note, student observations, learning updates, development observations and moments all hang off one occurrence. No duplicate class histories.
- AI drafts; humans approve.** Nothing AI-generated is persisted to a student record or the schedule until a human explicitly confirms. Parents never see a draft.

Delivery is **13 modules**, one at a time, each with schema → API → UI → tests, gated on your approval.

1. Requirements

1.1 Functional scope (in MVP)

#	Capability	Role	Source
F1	Username/password login, no OTP, forced first-login password change	All	Flow 08
F2	Basic settings (profile, password, notifications, language, timezone); role not user-editable	All	Flow 08
F3	Curriculum CRUD: Subject → Level → Topic → Skill, ordering, active/archive	Admin	Flow 03
F4	Level completion with impact preview + carry-forward	Admin	Flow 03/14
F5	Teacher management: capabilities (subject + level range), weekly availability, dated exceptions	Admin	Flow 10
F6	Student management: basic details, subjects & levels, weekly availability, parent access, status, archive	Admin	Flow 09
F7	Schedule week/day view, add class, recurrence	Admin	Flow 15
F8	Class scheduling: structured request → valid options (Best Match) → review → confirm. <i>Natural-language entry deferred — AI-PHASE-2</i>	Admin	Flow 06
F9	Rescheduling: select affected classes → proposed moves → confirm & apply. <i>Voice/NL entry deferred — AI-PHASE-2</i>	Admin	Flow 15
F10	Admin Home: today's classes + grouped Needs Attention + calm empty states	Admin	Flow 01/05
F11	Setup checklist in dependency order (Curriculum → Teachers → Students → Schedule)	Admin	Flow 04
F12	Teacher Home: today's classes, class records due, upcoming	Teacher	Flow 05/16
F13	Class record flow: open class → previous context → attendance → typed class note + per-student observations → optional updates → final review → atomic save. <i>Voice note + AI draft deferred — AI-PHASE-2</i>	Teacher	Flow 07/16
F14	Student Learning Map: per-student skill statuses (To Learn / Learning / Needs Support / Mastered) with note/evidence	Admin/Teacher	Flow 14
F15	Development: areas under Personality/Emotional/Physical, stages (Emerging/Developing/Consistent), evidence timeline	Admin/Teacher	Flow 13
F16	Moments: media gallery, add/tag, auto-filled class context, one media → many students	Admin/Teacher	Flow 12
F17	Parent Portal: Home, Learning, Development, Moments (read-only, linked child only)	Parent	Flow 11
F18	Weekly Update: Overview / Learning / Development / Moments + teacher note + PDF	Parent	Flow 02
F19	Notifications: parent 1/week; teacher scheduling + class-record; admin operational issues + password-reset requests	All	\$8
F20	Explicit empty, loading, error and offline states everywhere	All	Flow 04
F21	Forgot password (admin-mediated): user requests → admins notified → admin resets → temp password shown once to admin → shared out-of-band → forced change on next login	All	D2
F22	PWA: installable manifest, service worker, offline shell, push subscription registered (delivery behind FEATURE_WEB_PUSH)	All	D3

1.2 Explicitly out of scope (V2)

Parent schedule view · Ask Teacher / messaging · global search · library integration · advanced analytics · homework/assignments · OTP login · offline editing.

1.3 Deferred to Phase 2 — AI (D4)

Voice recording & transcription · AI class-note extraction · AI-proposed learning updates · AI-proposed development observations · natural-language class scheduling · voice rescheduling. Full inventory, integration seams and re-entry plan: `DEFERRED-AI.md`.

1.3 Non-functional

- Responsive desktop + phone from one codebase, matching the 16 boards. Reflow, do not miniaturize.
- Server-side role authorization on **every** endpoint (never trust the client role).
- Modern password hashing (delegated to Supabase Auth / bcrypt) + forced temp-password replacement.
- Object storage for media; DB stores references + metadata only.
- AI output is untrusted draft data until confirmed.
- Preserve author / source / timestamps on all learning, development and class-record history.
- WCAG AA contrast; status never encoded by colour alone; 44×44 px minimum touch targets.
- Optimise for small-school scale. No premature enterprise complexity.

2. Roles & permission matrix

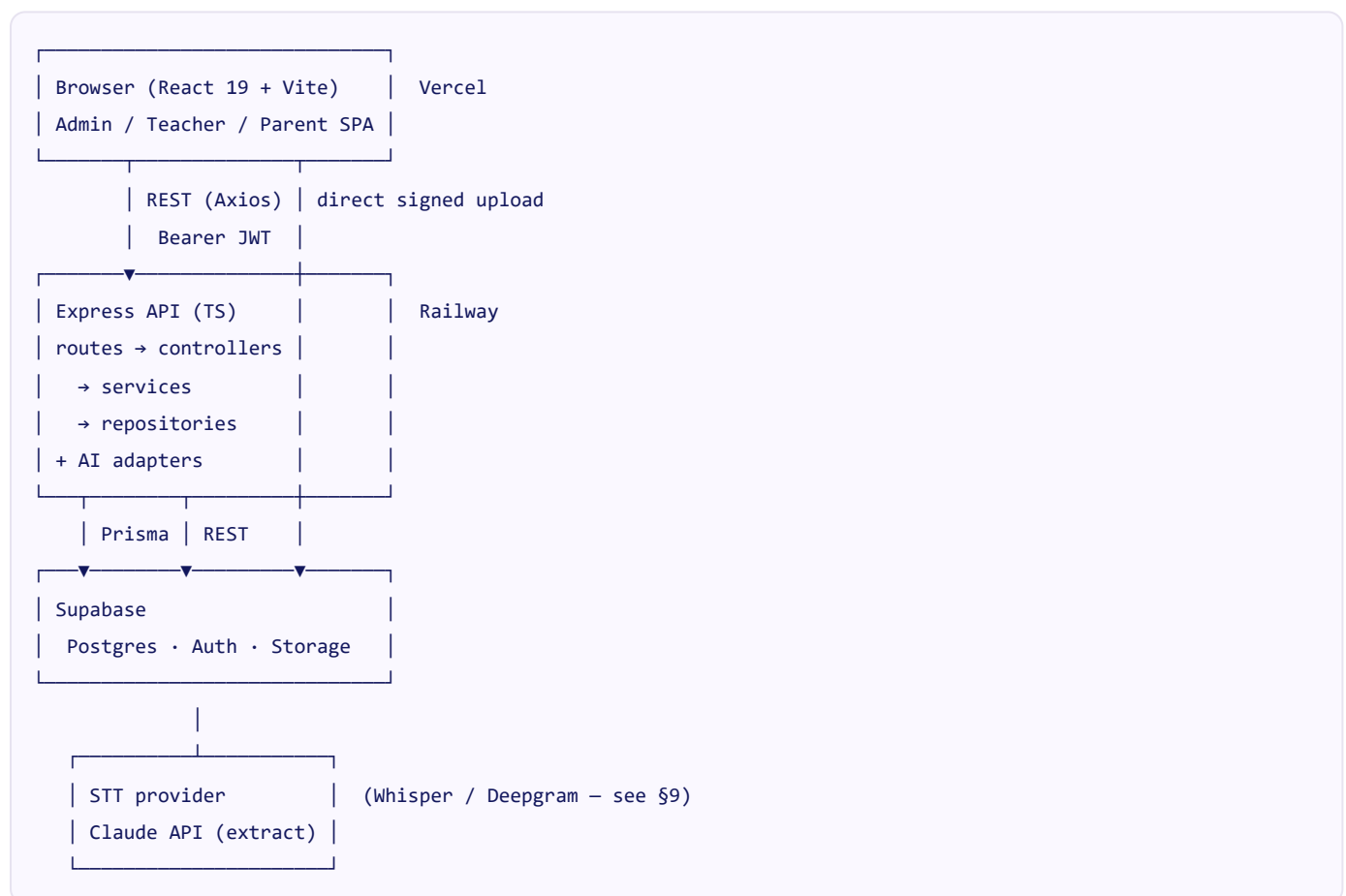
Resource	Admin	Teacher	Parent
Curriculum	CRUD	read	—
Teachers	CRUD	read self	—
Students	CRUD	read assigned	read linked child
Availability (teacher/student)	CRUD	read/edit own	—
Schedule / classes	CRUD	read own	—
Class occurrence + attendance	read all	CRUD own	—
Class record / voice note	read all	CRUD own	—
Learning Map	CRUD	update for own students	read linked child
Development	CRUD	add observation, update stage (own students)	read linked child
Moments	CRUD	create/tag own classes	read linked child
Weekly Update	read/regenerate	—	read linked child
Notifications	own	own	own
Settings	own	own	own

Enforcement: `requireRole()` + resource-scoped guards (`assertTeacherOwnsOccurrence` , `assertParentLinkedToStudent`). Parent and teacher scoping is applied in the **repository query**, not just a middleware check — no unscoped `findMany` on student data.

3. Business rules (locked)

BR-01 Class Occurrence is the anchor for attendance, class note, observations, learning updates, development observations and moments. **BR-02** Overall Class Note is always preserved, even when every optional update is skipped. **BR-03** AI never mutates schedule or records before explicit confirmation. **BR-04** No AI on dashboards, learning calculations, development stages, moments, notifications or auth. **BR-05** Availability is a constraint; the timetable is a booking; attendance is what actually happened. Three different things. **BR-06** Schedule validity = teacher capability $\wedge$ teacher availability $\wedge$ student availability $\wedge$ no conflict. Exceptions override recurring availability on their date. **BR-07** Curriculum defines what exists; the Learning Map defines where the student is. Skill status belongs to the student, not the curriculum. **BR-08** Level change preserves history: previous level stays queryable; unfinished skills may be carried forward. **BR-09** Learning/Development changes are append-only with author/date/source. Never destructive overwrite. **BR-10** Development is evidence over time, not a score. Stage change is human judgement, never automatic. **BR-11** One Moment = one media object with many student links. Never duplicate the media per student. **BR-12** Historical occurrences/records are immutable when future schedule, level or availability changes. **BR-13** Parent sees only approved outcomes for linked children. No drafts, no operational machinery. **BR-14** Parents: exactly one push per week. Teachers: scheduling + class-record only. Admin: actionable operational issues. **BR-15** Absent students must not receive AI-generated observations. **BR-16** Grouped Needs Attention: one root cause = one issue, with drill-down to affected classes. **BR-17** Archive, never hard-delete, anything with history. **BR-18** Signing the Parent Agreement is media consent. No separate consent form or per-moment consent gate. **BR-19** Class-record save is atomic across all its parts, or leaves a recoverable failure state. **BR-20** Dates/times are explicit and rendered in the configured school timezone.

4. System architecture



4.1 Key architectural decisions

AD-01 — Express is the only writer to Postgres. The frontend never talks to Supabase's data API. Prisma connects via the pooler (:6543 , pgbouncer) with a `DIRECT_URL` (:5432) for migrations. RLS stays *enabled with deny-all* on every table as defence-in-depth; the Supabase data API is disabled. Authorization lives in the service layer where the business rules already are (spec §9: "server-side role authorization").

AD-02 — Supabase Auth with a server-mediated username login. The spec locks username + password with no OTP; Supabase Auth is email-keyed. Resolution: each user row stores a `username` and a synthetic, non-routable email (<username>@users.littlepath.internal). `POST /auth/login` takes {username, password}, resolves username → email in `users`, calls `signInWithPassword` **server-side**, and returns the session. Benefits: generic invalid-login error (no user enumeration), `must_change_password` gate enforced before any other route, role never trusted from the client.

AD-03 — JWT verification via JWKS. Express verifies the Supabase access token with `jose` against the project JWKS, then loads the user row (short-TTL in-process cache) for role + status. Role is *not* read from the token alone — a disabled user is rejected on the next request, not on token expiry.

AD-04 — Media never proxies through the API. Upload: client asks Express for a signed upload URL (after an authz check) → uploads directly to a **private** Supabase Storage bucket → confirms to Express, which writes the metadata row. Read: Express issues short-lived signed URLs only after verifying the caller may see that student.

AD-05 — Occurrences are materialised. A class row holds the recurrence; a background job materialises `class_occurrences` for a rolling 120-day horizon and extends nightly. Materialised rows are what attendance, records and the schedule grid read. Past occurrences are immutable; editing a class regenerates only future, un-recorded ones.

AD-06 — Needs Attention is derived, not stored. A pure `AttentionService` recomputes grouped issues from current state on each Home request (cheap at this scale) and returns a deterministic `groupBy` per root cause. Nothing to keep in sync; an issue disappears the moment its cause is resolved. Only *dismissals* (if we add them) would be persisted.

AD-07 — The constraint engine is a pure function. `findValidSlots(request, snapshot) → SlotOption[]` takes an in-memory snapshot (capabilities, availabilities, exceptions, existing occurrences) and does no I/O. Fully unit-testable, deterministic, and reusable for both "find options" and "validate a proposed move".

AD-08 — AI behind interfaces (Phase 2 seam, built now). `ITranscriptionProvider`, `IClassNoteExtractor`, `ISchedulingInterpreter` are defined in this build with `Manual*` implementations that pass through operator-entered data. Phase 2 swaps the implementation in the DI container — no call-site changes. `ai_usage_log` and `GET /admin/ai-usage` ship now and stay empty until then.

AD-09 — Forgot password is an operational issue, not an auth channel (D2). `POST /auth/forgot-password` is unauthenticated, strictly rate-limited (5/hour/IP), and always returns the same generic success regardless of whether the username exists — no user enumeration. On a real match it inserts a `PASSWORD_RESET_REQUEST` notification for every active admin and surfaces the request in Needs Attention. `POST /admin/users/:id/reset-password` generates a temporary password, returns it **once** in the response body (never stored in plaintext, never logged, never re-retrievable), and sets `must_change_password = true`. The admin passes it to the user out-of-band.

5. Database schema (PostgreSQL / Prisma)

5.1 Enums

Role	ADMIN TEACHER PARENT
UserStatus	ACTIVE INACTIVE ARCHIVED
StudentStatus	ACTIVE INACTIVE ARCHIVED
CurriculumStatus	ACTIVE REVIEW ARCHIVED
SkillStatus	TO_LEARN LEARNING NEEDS_SUPPORT MASTERED
AttendanceStatus	PRESENT ABSENT LATE
DevCategory	PERSONALITY EMOTIONAL PHYSICAL
DevStage	EMERGING DEVELOPING CONSISTENT
OccurrenceStatus	SCHEDULED COMPLETED CANCELLED
ClassRecordStatus	DRAFT TRANSCRIBING PROCESSING IN_REVIEW SAVED FAILED
UpdateSource	CLASS_RECORD TEACHER_MANUAL ADMIN_MANUAL
NotificationType	WEEKLY_UPDATE_READY SCHEDULE_CHANGED CLASS_RECORD_DUE AVAILABILITY_CONFLICT TEACHER_AVAILABILITY_CHANGE STUDENT_AVAILABILITY_CHANGE INCOMPLETE_SETUP
AttentionType	TEACHER_UNAVAILABLE STUDENT_UNAVAILABLE SCHEDULE_CONFLICT INCOMPLETE_STUDENT_SETUP CLASS_RECORD_OVERDUE SETUP_INCOMPLETE
AiWorkflow	VOICE_TRANSCRIPTION VOICE_EXTRACTION SCHEDULE_INTERPRET SCHEDULE_CHANGE

5.2 Tables

Identity

- `school_settings` — singleton: name, timezone (`Asia/Kolkata`), week_start_day, weekly_update_day, weekly_update_time, ai_monthly_budget_cents
- `users` — id (= Supabase auth uid), username ^u, email_alias ^u, role, status, full_name, avatar_path, must_change_password, language, created_at
- `teachers` — id, user_id ^u, notes
- `parents` — id, user_id ^u
- `parent_students` — parent_id, student_id, relationship · PK(parent_id, student_id)
- `students` — id, full_name, date_of_birth, grade_label, joined_at, status, avatar_path, notes

Curriculum

- `subjects` — id, name ^u, color_token, display_order, status
- `levels` — id, subject_id, name, display_order, status · ^u(subject_id, name)
- `topics` — id, level_id, name, display_order, status
- `skills` — id, topic_id, name, description, learning_goal, display_order, status

Capability & availability

- `teacher_capabilities` — id, teacher_id, subject_id, min_level_order, max_level_order, is_primary
- `teacher_availability` — id, teacher_id, weekday(0–6), start_time, end_time
- `teacher_availability_exceptions` — id, teacher_id, date, is_available, all_day, start_time?, end_time?, reason
- `student_availability` — id, student_id, weekday, start_time, end_time

Student learning state

- `student_subject_levels` — id, student_id, subject_id, level_id, is_current, started_at, completed_at · ^u(student_id, subject_id) where is_current

- `student_skill_progress` — id, student_id, skill_id, status, updated_by, updated_at · ^u(student_id, skill_id) (*current state*)
- `learning_updates` — id, student_id, skill_id, previous_status, new_status, note, evidence_path?, source, class_record_id?, author_id, created_at (*append-only history*)
- `level_completions` — id, student_id, subject_id, from_level_id, to_level_id?, carried_forward_skill_ids[], confirmed_by, confirmed_at

Scheduling

- `classes` — id, subject_id, level_id, teacher_id, days_of_week[], start_time, duration_minutes, start_date, end_date?, timezone, status, created_by
- `class_students` — class_id, student_id · PK both
- `class_occurrences` — id, class_id, scheduled_start ^{tz}, scheduled_end ^{tz}, teacher_id (snapshot), status, cancelled_reason · ^u(class_id, scheduled_start) · idx(scheduled_start), idx(teacher_id, scheduled_start)
- `attendance` — id, occurrence_id, student_id, status, note · ^u(occurrence_id, student_id)
- `scheduling_requests` — id, admin_id, raw_text, interpreted (jsonb), status, created_at
- `schedule_change_proposals` — id, request_id, moves (jsonb), status, applied_at, applied_by

Class record

- `class_records` — id, occurrence_id ^u, author_id, overall_class_note, transcript, audio_path, language_detected, status, ai_model, ai_cost_micros, created_at, saved_at
- `student_observations` — id, class_record_id, student_id, observation, is_ai_generated, was_edited

Development

- `development_areas` — id, category, name, description, display_order, status (*school catalogue*)
- `student_development_areas` — id, student_id, area_id, current_stage, updated_at · ^u(student_id, area_id)
- `development_observations` — id, student_id, area_id, observation, observed_on, observer_id, class_record_id?, source, created_at
- `development_stage_changes` — id, student_id, area_id, from_stage, to_stage, changed_by, observation_id?, created_at

Moments

- `moments` — id, title, caption, subject_id?, class_occurrence_id?, captured_on, created_by, source
- `moment_media` — id, moment_id, storage_path, thumbnail_path, mime_type, width, height, duration_ms, size_bytes, display_order
- `moment_students` — moment_id, student_id · PK both

Parent-facing & system

- `weekly_updates` — id, student_id, week_start (date), week_end, summary_text, teacher_note, status (DRAFT|PUBLISHED), generated_at, published_at · ^u(student_id, week_start)
- `weekly_update_items` — id, weekly_update_id, item_type (LEARNING|DEVELOPMENT|MOMENT|CLASS_NOTE), ref_id, highlight_text, display_order
- `notifications` — id, recipient_user_id, type, title, body, payload (jsonb), read_at, created_at · idx(recipient_user_id, created_at)
- `ai_usage_log` — id, workflow, provider, model, input_tokens, output_tokens, audio_seconds, cost_micros, occurrence_id?, user_id, created_at
- `audit_log` — id, actor_id, action, entity, entity_id, before (jsonb), after (jsonb), created_at

^u = unique · ^{tz} = `timestampztz` (UTC stored, school timezone rendered)

5.3 Integrity notes

- Every history table (`learning_updates` , `development_observations` , `class_records`) is **insert-only**; corrections create a new row.
- `student_skill_progress` is a projection of `learning_updates` kept in the same transaction — fast reads, replayable truth.
- `class_occurrences` snapshots `teacher_id` so a later teacher reassignment does not rewrite history (BR-12).
- Deletes are soft everywhere history exists (`status = ARCHIVED`).

6. API design (`/api/v1`)

Envelope: `{ data, meta? }` on success; `{ error: { code, message, details? } }` on failure. Zod validates every body/query at the route boundary using schemas shared with the frontend.

Group	Endpoints
Auth	<code>POST /auth/login</code> · <code>POST /auth/logout</code> · <code>POST /auth/refresh</code> · <code>POST /auth/change-password</code> · <code>GET /auth/me</code>
Admin users	<code>POST /admin/users</code> (create with temp password) · <code>PATCH /admin/users/:id/status</code> · <code>POST /admin/users/:id/reset-password</code>
Curriculum	<code>GET/POST /subjects</code> · <code>GET/PATCH /subjects/:id</code> · <code>GET/POST /subjects/:id/levels</code> · <code>PATCH /levels/:id</code> · <code>GET/POST /levels/:id/topics</code> · <code>PATCH /topics/:id</code> · <code>GET/POST /topics/:id/skills</code> · <code>PATCH /skills/:id</code> · <code>PATCH /:parent/:id/reorder</code>
Teachers	<code>GET/POST /teachers</code> · <code>GET/PATCH /teachers/:id</code> · <code>GET/PUT /teachers/:id/capabilities</code> · <code>GET/PUT /teachers/:id/availability</code> · <code>GET/POST /teachers/:id/exceptions</code> · <code>DELETE /exceptions/:id</code>
Students	<code>GET/POST /students</code> · <code>GET/PATCH /students/:id</code> · <code>PUT /students/:id/subject-levels</code> · <code>PUT /students/:id/availability</code> · <code>PUT /students/:id/parent-access</code> · <code>PATCH /students/:id/status</code> · <code>GET /students/:id/history</code>
Learning	<code>GET /students/:id/learning</code> · <code>GET /students/:id/learning/:subjectId</code> · <code>PATCH /students/:id/skills/:skillId</code> · <code>GET /students/:id/level-change/preview</code> · <code>POST /students/:id/level-change</code>
Development	<code>GET/POST /development/areas</code> · <code>GET /students/:id/development</code> · <code>GET /students/:id/development/:areaId</code> · <code>POST /students/:id/development/:areaId/observations</code> · <code>PATCH /students/:id/development/:areaId/stage</code>
Moments	<code>GET /moments</code> (filters) · <code>POST /moments/upload-url</code> · <code>POST /moments</code> · <code>GET/PATCH /moments/:id</code> · <code>GET /students/:id/moments</code>
Schedule	<code>GET /schedule?from&to&view</code> · <code>POST /classes</code> · <code>GET/PATCH /classes/:id</code> · <code>POST /classes/:id/cancel</code> · <code>GET /occurrences/:id</code>
AI scheduling	<code>POST /schedule/interpret</code> · <code>POST /schedule/options</code> · <code>POST /schedule/confirm</code> · <code>POST /schedule/change/interpret</code> · <code>POST /schedule/change/apply</code>
Home / attention	<code>GET /home/admin</code> · <code>GET /home/teacher</code> · <code>GET /home/parent</code> · <code>GET /attention</code> · <code>GET /setup/progress</code>
Class record	<code>GET /teacher/occurrences?date</code> · <code>GET /occurrences/:id/context</code> · <code>PUT /occurrences/:id/attendance</code> · <code>POST /occurrences/:id/record</code> (creates DRAFT + signed audio upload URL) · <code>POST /class-records/:id/process</code> · <code>GET /class-records/:id</code> · <code>PATCH /class-records/:id</code> · <code>POST /class-records/:id/save</code>

Group	Endpoints
Parent	GET /parent/children · GET /parent/students/:id/learning · .../development · .../moments · GET /parent/weekly-updates · GET /parent/weekly-updates/:id · GET /parent/weekly-updates/:id/pdf
Notifications	GET /notifications · PATCH /notifications/:id/read · POST /notifications/read-all
Ops	GET /health · GET /admin/ai-usage

POST /class-records/:id/save is the one genuinely transactional endpoint: attendance + class note + observations + accepted learning updates + accepted development observations + moment links, all in one Prisma `$transaction`, or nothing (BR-19).

7. Frontend architecture

```
apps/web/src/
  app/           router, providers (QueryClient, Auth, Theme), error boundaries
  routes/       role-gated route trees: admin/ teacher/ parent/ auth/
  layouts/      AdminShell (left rail) · TeacherShell · ParentShell · MobileTabBar
  features/
    auth/ curriculum/ teachers/ students/ schedule/ class-record/
    learning/ development/ moments/ parent/ notifications/ home/
      └─ api/           typed hooks over Axios + TanStack Query
      └─ components/
      └─ hooks/
      └─ schemas/       re-exported from @littlepath/shared
      └─ pages/
  components/
    ui/          shadcn primitives
    patterns/    StatusChip, AttentionCard, EmptyState, LoadingState,
                  ErrorState, OfflineBanner, SubjectBadge, StageChip,
                  AvailabilityGrid, RecordStepper
  lib/          axios client + interceptors, queryClient, dates (tz), format, cn
  styles/       tokens.css (design-system §14) + tailwind theme
```

Design system → code. Section 14 tokens become CSS custom properties consumed by Tailwind v4 `@theme`:

```
@theme {
  --color-brand-navy: #11165C; --color-brand-violet: #5B2CCB;
  --color-brand-violet-hover: #4B22B5;
  --color-surface-canvas: #FCFBFF; --color-surface-card: #FFFFFF;
  --color-surface-lavender: #F3EEFF; --color-surface-lavender-2: #FAF8FF;
  --color-border-default: #E7E3F0;
  --color-text-primary: #11165C; --color-text-secondary: #5F6080;
  --color-text-muted: #8D8DA5;
  --color-success: #3FA45B; --color-success-bg: #EDF8EF;
  --color-info: #3F83E8; --color-info-bg: #EDF5FF;
  --color-warning: #F29A38; --color-warning-bg: #FFF5E8;
  --color-danger: #F05B5B; --color-danger-bg: #FFF0F0;
  --color-pink: #E95D86; --color-pink-bg: #FFF1F6;
  --radius-sm: 8px; --radius-md: 12px; --radius-lg: 16px;
  --radius-xl: 20px; --radius-2xl: 24px; --radius-pill: 999px;
  --font-display: Georgia, 'Times New Roman', serif;
  --font-sans: Inter, 'Segoe UI', system-ui, sans-serif;
}
```

Spacing uses Tailwind's native 4px scale (`space.1` → `p-1` ... `space.12` → `p-12`), so no custom spacing tokens are needed.

Enforced UI rules. Subject colour mapping is a single constant (`Mathematics` → `violet`, `English` → `orange`, `Science` → `blue`, `Telugu` → `green`). Learning-state → colour is a single constant. `StatusChip` always renders icon + label + colour, so status survives grayscale (§10). Serif is reserved for `Display/H1/H2/H3` ; a lint rule keeps it out of table cells and metadata.

Repo layout (pnpm workspaces + Turborepo):

```
littlepath/
  apps/web/      React 19 + Vite
  apps/api/      Express + TS
  packages/shared/  Zod schemas, DTO types, enums, constants (single source for both sides)
  packages/config/  eslint, tsconfig, prettier, tailwind preset
  prisma/        schema.prisma, migrations, seed.ts
  docs/
```

`apps/api` layering: `routes` → `controllers` → `services` → `repositories` → `prisma` . Controllers do no business logic; services do no HTTP; repositories do no rules. Dependency injection via a small container so services take interfaces (`IStorageProvider` , `ITranscriptionProvider` , `IClock`) — makes the constraint engine and the class-record save path testable without a database.

8. AI architecture — ⚠️ DEFERRED TO PHASE 2 (D4)

Nothing in this section is built in the one-week delivery. It is retained as the Phase-2 specification. What ships instead: the teacher types the class note and per-student observations directly, and the admin fills a structured scheduling form. Both paths write through the *same* services and endpoints AI will later feed, so Phase 2 is an input-source change, not a rework. See `DEFERRED-AI.md` for the seam-by-seam inventory.

8.1 Voice → class record

```
audio (client) -signed URL→ Supabase Storage
|
|1- ITranscriptionProvider.transcribe(path) → transcript + language
|   Whisper (or Deepgram) – must handle English / Telugu / Hindi
|
|2- IClassNoteExtractor.extract({ transcript, roster, attendance,
|                               subject, level, topics, skills })
|   Claude, structured output (Zod → JSON schema, strict)
|   → { overallClassNote,
|       studentObservations[{studentId, observation}],
|       proposedLearningUpdates[{studentId, skillId, newStatus, rationale}],
|       proposedDevelopmentObservations[{studentId, areaId, observation}] }
```

Guardrails: absent students are removed from the roster passed to the model **and** filtered from the output (BR-15). Skill IDs are validated against the class's actual level. Every field lands in the review UI as editable/removable. Nothing persists until `POST /class-records/:id/save`.

8.2 Scheduling

`ISchedulingInterpreter` converts free text → `{ studentIds, subjectId, teacherId?, frequency, durationMinutes, timePreference, startDate }` with a confidence flag. **The AI only parses.** `SchedulingEngine.findValidSlots()` — pure, deterministic code — produces the options against capabilities, availabilities, exceptions and existing occurrences. Same split for rescheduling: AI identifies intent + affected classes; the engine validates every proposed move; admin confirms.

8.3 Budget

Model IDs and prices (per million tokens):

Model	ID	Input	Output
Claude Opus 5	<code>claude-opus-5</code>	\$5	\$25
Claude Sonnet 5	<code>claude-sonnet-5</code>	\$3 (\$2 intro to 2026-08-31)	\$15 (\$10 intro)
Claude Haiku 4.5	<code>claude-haiku-4-5</code>	\$1	\$5

Claude has **no speech-to-text** — STT is a separate provider (Whisper ≈ \$0.006/min, or Deepgram for stronger Telugu/Hindi code-switching).

Estimate at 5 teachers × 4 classes/day × 22 days ≈ **440 class records/month**, ~2 min audio each, ~3,000 input / ~1,200 output tokens per extraction:

Configuration	STT	Extraction	Scheduling	
Opus 5	~\$6	~\$20	~\$2	~\$28
Sonnet 5 (intro)	~\$6	~\$8	~\$1	~\$15
Haiku 4.5	~\$6	~\$4	~\$1	~\$11

Recommendation: Opus 5. The extraction writes into a child's permanent academic record, and the cost fits the stated \$30 budget — but with no headroom, so this is a decision for you (Q6). Prompt caching on the system prompt + curriculum context (512-token minimum on Opus 5) trims input cost further. `ai_usage_log` + `GET /admin/ai-usage` make spend visible, with a soft-alert at 80% and a configurable hard stop.

9. Module delivery plan

Each module ships: Prisma migration → service + repository → Zod contracts in `shared` → API routes → React feature → unit tests on rules → manual verification against its board. **You approve before I start the next one.**

#	Module	Depends on	Flows
M0	Foundation: monorepo, Prisma, Supabase, CI, tokens, shells, system-state components	—	04
M1	Auth & settings: login, forced password change, JWT middleware, RBAC, admin user creation, forgot-password → admin reset loop	M0	08
M2	Curriculum: Subject→Level→Topic→Skill, ordering, archive	M1	03
M3	Teachers: profile, capabilities, availability, exceptions	M1	10
M4	Students: add wizard, profile tabs, subject levels, availability, parent access, status	M2, M3	09
M5	Scheduling: constraint engine, calendar, classes, occurrence materialisation	M3, M4	15
M6	Home & Needs Attention: admin/teacher home, attention engine, setup checklist	M5	01, 05, 04
M7	Class record (manual): context, attendance, typed class note, per-student observations, optional updates, final review, atomic save. <i>AI draft seam stubbed</i>	M5, M6	07, 16
M8	Learning Map: skill statuses, level completion, carry-forward	M2, M4, M7	14
M9	Development: areas, observations, stage changes, evidence timeline	M4, M7	13
M10	Moments: upload, tagging, gallery, class auto-context	M4, M7	12
M11	Reschedule flow: pick affected classes, engine revalidates, proposed-moves review, confirm & apply. <i>NL/voice entry stubbed</i>	M5	06, 15
M12	Parent portal + weekly update: home, learning, development, moments, weekly update, PDF	M8, M9, M10	02, 11
M13	Notifications, polish, deployment: rules, in-app centre, a11y pass, Vercel + Railway	all	08 \$8

10. One-week roadmap

Hour-by-hour, two-track daily schedule: `WEEK-1-SCHEDULE.md`.

Summary — Day 1 Foundation + Auth · Day 2 Auth complete + Curriculum · Day 3 Teachers + Students · Day 4 Scheduling engine + calendar · Day 5 Home/Needs Attention + Class Record · Day 6 Learning Map + Development + Moments · Day 7 Parent portal + Weekly Update + Notifications + PWA + deploy.

11. Testing & acceptance

- **Unit (Vitest):** constraint engine, attention grouping, recurrence expansion, level-completion carry-forward, weekly-update selection, permission guards. These encode the business rules and are the regression net.
- **Integration (Supertest + test DB):** the atomic class-record save (including a deliberate mid-save failure → recoverable state), parent scoping, forced password change gate.
- **E2E (Playwright):** the four stories from Part IV of the business document — new school setup, one class through the whole product, what the parent sees, teacher becomes unavailable.

- **Acceptance:** the 10-item MVP checklist in Developer Spec §11 is the definition of done.

12. Open questions

Each has a **default I will proceed with** if you don't answer — no question blocks the start of M0/M1.

#	Question	My default
Q1	ANSWERED (D1). Product name is Valmiki LMS System .	Logo artwork still needed — placeholder mark until supplied.
Q2	ANSWERED (D2). Forgot password notifies admins; admin resets and shares the password manually. No email/SMS.	Implemented as F21 / AD-09.
Q3	ANSWERED (D3). Responsive web + PWA + service worker + VAPID built now; push <i>delivery</i> behind FEATURE_WEB_PUSH as the flagged follow-up.	Implemented as F22 / M13.
Q4	"Week at a glance" narrative. The parent board shows a written paragraph. Spec §5 approves AI for <i>only</i> voice and scheduling. Is that paragraph AI-written or assembled?	Templated assembly from approved learning/development items. Cheap, deterministic, in-scope. Say the word and it becomes a third AI workflow (~+\$3/mo).
Q5	Weekly update publishing. Auto-generated on a schedule, or does a human review before parents see it? Who writes the teacher note?	Auto-generate Friday 18:00 Asia/Kolkata as DRAFT , admin publishes (which fires the single weekly push). Teacher note is optional, added by the class teacher or admin before publish.
Q6	DEFERRED (D4). AI model choice moves to the Phase-2 kickoff. Cost table retained in §8.3.	—
Q7	DEFERRED (D4). Voice languages & audio retention move to Phase 2.	—
Q8	Multi-child parents. The parent home shows one child card. Can a parent be linked to several students?	Yes — schema supports it; UI adds a child switcher in the parent header.
Q9	Recurrence complexity. Fixed weekly pattern (days + time + duration), or full RRULE with terms/holidays?	Fixed weekly pattern with a start/end date. No holiday calendar in MVP; admin cancels individual occurrences.
Q10	Editing a saved class record. Allowed after save? Time-limited?	Editable by the authoring teacher for 48 hours, then admin-only; every edit appends to audit_log and never rewrites the original learning/development rows.
Q11	Development areas catalogue. Boards show Confidence, Independence, Curiosity, Communication, Collaboration, Resilience, Self-regulation. Fixed seed or admin-editable?	Seeded from the boards, admin-editable (add/rename/archive) — costs little and avoids a migration later.
Q12	Level progression authority. "An authorized user" moves a student to the next level. Admin only, or teachers too?	Admin only. Teachers update skill statuses; level change is a structural decision.
Q13	PDF weekly report. In MVP? Server-side (Puppeteer on Railway, +memory) or client-side (react-pdf)?	In MVP, client-side react-pdf — no headless-Chrome footprint on Railway, and the data is already loaded in the page.
Q14	Typography. The design system names Aptos, which isn't reliably available on the web.	Inter as the sans (the boards' rendering is consistent with it), Georgia as the serif per the spec.
Q15	Environments. One Supabase project or separate staging/production? Who owns the Vercel/Railway/Supabase accounts?	Two Supabase projects (staging + production), Vercel preview deploys per PR, Railway staging + production services. Need your accounts/credentials before M13.

13. What I need from you

Before Day 1 starts:

1. **Supabase** — project created (or permission for me to create one) + connection strings and service key.
2. **Logo artwork** for Valmiki LMS System. A placeholder wordmark ships until it arrives; swapping it later is a 5-minute change.

Before Day 7 (deploy):

1. **Vercel + Railway accounts** (or permission to create under your org).

Not needed at all this week: Anthropic API key, STT provider key — deferred with the rest of Phase 2.

Remaining open questions Q4, Q5, Q8–Q15 all have working defaults and do not block any day. Overrule any of them at any point in the week; none is expensive to change before Day 7.

Valmiki LMS System — One-Week Delivery Schedule

Window: Day 1 = Mon 27 Jul 2026 → Day 7 = Sun 2 Aug 2026 **Team assumed:** 2 engineers — BE (backend-leaning) and FE (frontend-leaning), working in parallel from Day 1. **Scope:** 16 flows, AI deferred (see `DEFERRED-AI.md`).

⚠ **Days 6 and 7 land on Sat 1 / Sun 2 Aug.** If the team doesn't work weekends, shift to **Mon 3 / Tue 4 Aug** — the day contents are unchanged, the calendar just stretches to 9 elapsed days. Flag this now rather than on Friday.

⚠ **This is an aggressive schedule.** Seven working days for 16 flows, three role portals and a responsive PWA is tight even with AI removed. It is achievable because the schema is designed up front (§5 of the plan) and the two tracks never block each other — but it has no slack. §4 below is the cut list, in the order I'd cut.

Daily rhythm

Time	
09:00–09:15	Standup — yesterday's exit criteria confirmed, today's contracts agreed
09:15–13:00	Block A (3h45)
13:00–14:00	Break
14:00–17:30	Block B (3h30)
17:30–18:00	Integration + demo checkpoint against the day's exit criteria

Contract-first rule. BE's first task each morning is to publish that day's Zod schemas and DTO types into `packages/shared`. FE codes against those types immediately, with MSW mocks if the endpoint isn't live yet. This is what keeps the two tracks from blocking each other — it is not optional.

Day 1 — Mon 27 Jul • Foundation + Auth backbone

Modules: M0, M1 (start)

BE

Time	Task	Output
09:15–10:45	pnpm workspace + Turborepo; <code>apps/api</code> , <code>apps/web</code> , <code>packages/shared</code> , <code>packages/config</code> ; shared tsconfig / eslint / prettier	Repo builds, <code>pnpm dev</code> runs both apps
10:45–13:00	Supabase staging project; Prisma init (pooler + <code>DIRECT_URL</code>); schema part 1 — <code>school_settings</code> , <code>users</code> , <code>teachers</code> , <code>parents</code> , <code>parent_students</code> , <code>students</code> , <code>subjects</code> , <code>levels</code> , <code>topics</code> , <code>skills</code>	First migration applied
14:00–15:45	Schema part 2 — <code>capabilities</code> , <code>availabilities</code> , <code>exceptions</code> , <code>student_subject_levels</code> , <code>student_skill_progress</code> , <code>learning_updates</code> , <code>level_completions</code> , <code>classes</code> , <code>class_students</code> , <code>class_occurrences</code> , <code>attendance</code>	Second migration applied

Time	Task	Output
15:45–17:30	Schema part 3 — class_records, student_observations, development_*, moments_*, weekly_updates_*, notifications, ai_usage_log, audit_log; <code>seed.ts</code> (school settings + 1 admin)	Full schema live; RLS deny-all enabled; <code>GET /health</code> green

FE

Time	Task	Output
09:15–10:30	Vite + React 19 + TS + Tailwind v4 + shadcn init; path aliases	App boots
10:30–13:00	<code>tokens.css</code> from Design System §14; Tailwind <code>@theme</code> ; typography scale (Georgia display / Inter sans); subject-colour and learning-state constant maps	Design tokens usable everywhere
14:00–15:45	System-state components: <code>EmptyState</code> , <code>LoadingState</code> (skeletons), <code>ErrorState</code> , <code>OfflineBanner</code> , <code>StatusChip</code> (icon + label + colour), <code>SubjectBadge</code> , <code>StageChip</code>	Component gallery route renders all states
15:45–17:30	React Router role-gated trees; <code>AdminShell</code> (left rail), <code>TeacherShell</code> , <code>ParentShell</code> , <code>MobileTabBar</code> ; Axios client + interceptors; TanStack Query provider	Shells render at 375 / 768 / 1280 px

Exit criteria: repo builds and runs; full database schema migrated; three role shells render responsively with placeholder pages; health check green.

Day 2 — Tue 28 Jul • Auth complete + Curriculum

Modules: M1 (finish), M2

BE

Time	Task	Output
09:15–11:15	Express middleware stack: JWKS JWT verify, <code>requireRole</code> , resource guards, Zod validate, error handler, rate limiter, helmet. Auth service: username→email resolve, login, logout, refresh, <code>/auth/me</code>	Login returns a session; role enforced server-side
11:15–13:00	<code>must_change_password</code> gate (blocks all non-auth routes); <code>POST /auth/change-password</code> ; <code>POST /admin/users</code> (temp password); <code>POST /auth/forgot-password</code> (rate-limited, generic response, raises <code>PASSWORD_RESET_REQUEST</code> to all admins); <code>POST /admin/users/:id/reset-password</code> (temp password returned once)	D2 loop working end-to-end
14:00–15:45	Curriculum service + repository: subjects → levels → topics → skills CRUD, <code>display_order</code> reorder, archive semantics	Contracts published to <code>shared</code>
15:45–17:30	Curriculum routes; unit tests: ordering, archive-not-delete, level-name uniqueness per subject	Curriculum API complete

FE

Time	Task	Output
09:15–11:00	Login page (username, password visibility toggle, generic invalid-login error, "Need help accessing your account?"); Forgot Password screen + confirmation state	Flow 08 screens 1–4
11:00–13:00	First-login Create a new password (strength meter, confirm, requirements hint); Settings shell — Profile / Change Password / Notifications / Language / Time Zone; role shown read-only	Flow 08 screens 5–6

Time	Task	Output
14:00–15:45	Curriculum Home (subject cards) + Subject Overview (levels list) + Add Subject / Add Level dialogs	Flow 03 screens 1–2
15:45–17:30	Level Overview (topics), Topic & Skills view, Skill Detail edit sheet, reorder UI	Flow 03 screens 3–5

Exit criteria: admin logs in with a temp password, is forced to change it, and builds a complete curriculum (Mathematics → Level 6 → Fractions → 4 skills). A parent taps Forgot Password and the admin sees the request in their notifications.

Day 3 — Wed 29 Jul • Teachers + Students

Modules: M3, M4

BE

Time	Task	Output
09:15–11:00	Teacher CRUD; teaching capabilities (subject + min/max level order); capability→level resolution helper	Contracts published
11:00–13:00	Regular weekly availability; dated exceptions; <code>resolveAvailability(person, date)</code> — exception overrides recurring — with its unit test suite	The rule the scheduler depends on, proven
14:00–15:45	Student CRUD, subject/level assignment (per-subject current level), weekly availability	Contracts published
15:45–17:30	Parent access linking, status/archive, student profile aggregate endpoint (<code>GET /students/:id</code> → overview + current levels + parent)	Flow 09 API complete

FE

Time	Task	Output
09:15–11:00	Teachers list; Teacher Profile Overview (teaching summary, today, upcoming exceptions)	Flow 10 screens 1–2
11:00–13:00	Teaching Capabilities editor; Weekly Availability grid (day toggles + time ranges); Add Exception dialog; save confirmation	Flow 10 screens 3–6
14:00–15:45	Students list (with per-subject level chips); Student Profile Overview tab; Manage Student menu	Flow 09 screens 1–3
15:45–17:30	Add Student wizard — 5 steps: Basic Details → Subjects & Levels → Weekly Availability → Parent Access → Review & Create	Flow 09 screens 4–6

Exit criteria: a teacher exists with capabilities, a weekly availability pattern and a one-day exception; a student exists with per-subject levels, availability and a linked parent account. The scheduler now has real data to work with.

Day 4 — Thu 30 Jul • Scheduling engine + calendar

Modules: M5 • highest-risk day

BE

Time	Task	Output
09:15–12:00	Constraint engine (pure, no I/O): capability filter → availability intersection → exception override → existing-occurrence conflict → recurring pattern search → Best Match ranking. Written test-first.	<code>findValidSlots()</code> + ~25 unit tests green
12:00–13:00	Occurrence materialisation: expand recurrence over a 120-day horizon; nightly extend job; idempotent on <code>(class_id, scheduled_start)</code>	Occurrences generated
14:00–15:30	Classes CRUD; <code>class_students</code> ; cancel-occurrence; edit regenerates future un-recorded occurrences only (BR-12)	Flow 15 write API
15:30–17:30	<code>GET /schedule?from&to&view</code> ; <code>GET /occurrences/:id</code> ; <code>POST /schedule/options</code> (structured request → ranked slots)	Flow 06/15 read API

FE

Time	Task	Output
09:15–11:00	Schedule week grid — time axis, day columns, subject-coloured class blocks, timezone footer	Flow 15 screen 1
11:00–13:00	Day view , date navigation, Today, week/month/list toggle	Flow 15 screen 2
14:00–15:30	Add Class wizard step 1 — structured request form (students, subject, level, teacher, frequency, duration, time preference, start date), wrapped in <code>SchedulingInputPanel mode="form"</code>	Flow 06 screens 2–3 substitute
15:30–17:30	Slot options list with Best Match badge and per-option constraint checks; Review & Confirm; Class Scheduled success screen	Flow 06 screens 4–7

Exit criteria: admin requests "Mathematics, Aarav + Priya, twice a week, mornings, 1 hour", gets ranked valid options that respect capability + both availabilities + the exception, confirms one, and sees the recurring class on the week grid.

Day 5 — Fri 31 Jul • Home, Needs Attention, Class Record

Modules: M6, M7

BE

Time	Task	Output
09:15–11:00	Attention engine: teacher-unavailable, student-unavailable, schedule conflict, incomplete student setup, class-record overdue, setup incomplete → grouped by deterministic <code>groupBy</code> (BR-16) + unit tests	<code>GET /attention</code>
11:00–13:00	<code>GET /home/admin</code> , <code>GET /home/teacher</code> , <code>GET /setup/progress</code> (dependency-ordered checklist)	Flow 01/04/05 API
14:00–15:30	<code>GET /occurrences/:id/context</code> (roster + previous class record); <code>PUT /occurrences/:id/attendance</code> ; class-record draft create + PATCH	Flow 07/16 API part 1
15:30–17:30	<code>POST /class-records/:id/save</code> — one Prisma <code>\$transaction</code> : attendance + class note + observations + learning updates (+ <code>student_skill_progress</code> projection) + development observations + moment links. Integration test including a forced mid-save failure → recoverable state (BR-19)	The product's spine, tested

FE

Time	Task	Output
09:15–11:00	Admin Home: today's classes timeline + Needs Attention cards (grouped, with drill-down) + "Everything looks good" + "No classes scheduled today"	Flow 01 screens 1–4, Flow 05
11:00–13:00	Brand-new / partially-configured Home with setup checklist ; Teacher Home (today, class records due, upcoming)	Flow 04 screens 1–3, Flow 05 screen 5
14:00–15:30	Open Class / before-class context (students, previous class record); Attendance screen (Present / Absent / Late per student, running tally, optional note)	Flow 07 screens 1–2, Flow 16 screens 2–3
15:30–17:30	Class-record stepper : Overall Class Note (typed, required) → per-student Observations (typed, optional); RecordSourceStep placeholder slot mounted but empty	Flow 07/16, AI-3 manual substitute

Exit criteria: teacher opens today's class, sees who's in it and what happened last time, marks attendance, types the class note and two student observations, and saves. Admin Home shows a grouped "Priya is unavailable — 3 classes affected" issue that drills through to the affected classes.

Day 6 — Sat 1 Aug • Learning Map, Development, Moments

Modules: M8, M9, M10

BE

Time	Task	Output
09:15–10:30	Learning Map read; skill status update — append learning_updates + project student_skill_progress in one transaction (BR-09)	Flow 14 API
10:30–12:00	Level completion preview (mastered / remaining counts) + confirm + carry-forward + history preservation (BR-08) + unit tests	Flow 03 screen 6, Flow 14 screen 6
12:00–13:00	Development: areas catalogue (seeded from boards, admin-editable), observations, stage change + development_stage_changes audit	Flow 13 API
14:00–15:30	Moments: signed upload URL, create moment, tag many students to one media object (BR-11), gallery query with filters, signed read URLs	Flow 12 API
15:30–16:45	Distribution wiring: class-record save → learning updates + development observations + moment links land in the right places (Flow 07 §C)	One save updates three surfaces
16:45–17:30	Parent read endpoints, scoped in the repository layer: children, learning, development, moments	Flow 11 API

FE

Time	Task	Output
09:15–11:00	Student Learning Map: subject tabs → current level → topics → skills with status chips; status distribution summary	Flow 14 screens 2–4
11:00–13:00	Edit Skill sheet (status radio, note, evidence); Move to Level 7 modal (level summary, carry-forward choice, confirm)	Flow 14 screens 5–6
14:00–15:30	Development: student overview (areas by category with stages), Area Detail evidence history, Add Observation, Update Stage (optional)	Flow 13 screens 1–6

Time	Task	Output
15:30–16:45	Moments gallery/grid, Add Moment (upload → select students → title/caption → auto-filled context), Moment Detail	Flow 12 screens 1–6
16:45–17:30	Class-record Optional Updates step (add learning update / add development observation / attach media) + Final Review + Saved confirmation	Flow 07 screens 6–8

Exit criteria: one saved class record demonstrably updates the student's Learning Map, appends a dated Development observation, and creates a tagged Moment — without the teacher visiting three modules.

Day 7 — Sun 2 Aug · Parent portal, Weekly Update, Notifications, PWA, deploy

Modules: M12, M13

BE

Time	Task	Output
09:15–10:30	Weekly Update generator: item selection from approved records + templated Week at a Glance; Friday 18:00 Asia/Kolkata job; DRAFT → PUBLISH	Flow 02 API
10:30–12:00	Notification rules (BR-14): parent 1/week on publish · teacher scheduling + record-due · admin operational + password-reset requests; list / mark-read endpoints	Flow 02 screens 4–5
12:00–13:00	VAPID keypair, push-subscription storage endpoints, send path behind <code>FEATURE_WEB_PUSH</code> (off)	D3 infrastructure
14:00–15:30	Railway deploy (staging + production): env config, migrations, CORS, rate limits, helmet, log redaction	API live
15:30–16:30	Demo seed data (curriculum, 2 teachers, 5 students, a week of classes and records); audit-log wiring	Demoable environment
16:30–18:00	Walk the MVP acceptance checklist (Dev Spec §11) + the four Part IV stories	Sign-off evidence

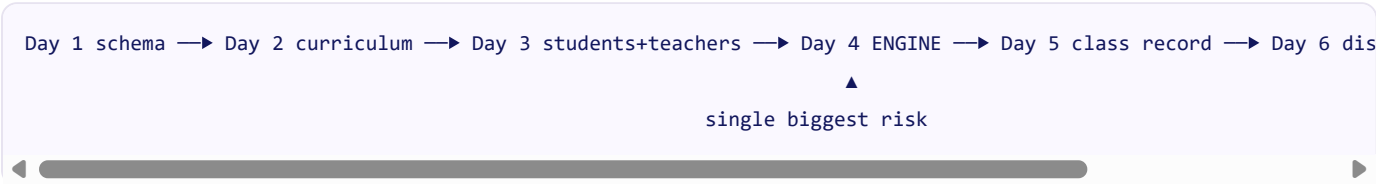
FE

Time	Task	Output
09:15–10:30	Parent Home: child card, weekly-update CTA, Learning Now, Development Snapshot, recent Moment, child switcher	Flow 11 screen 1, Flow 02 screen 2
10:30–12:00	Weekly Update: Overview / Learning / Development / Moments tabs + teacher note; Download PDF (react-pdf)	Flow 02 screen 3
12:00–13:00	Parent Learning / Development / Moments tabs (read-only)	Flow 11 screens 2–4
14:00–15:00	Notification centre + unread badges for all three roles	Flow 02 screens 4–5
15:00–16:00	PWA: manifest, icon set, Workbox service worker, offline shell, install prompt, push-subscription registration	D3 / F22
16:00–17:00	Accessibility + responsive pass: 44×44 targets, WCAG AA contrast, keyboard nav, focus rings, reflow at 375 / 768 / 1280	\$10 compliance

Time	Task	Output
17:00–18:00	Vercel deploy + smoke test of the four stories on production	Shipped

Exit criteria: the full cycle runs on staging — admin sets up the school, schedules a class, teacher records it, parent receives and reads the weekly update — and the MVP acceptance checklist is green.

2. Critical path



Everything downstream of **Day 4's constraint engine** depends on it. It is scheduled test-first in the freshest block of the day for that reason. If it isn't green by 12:00 Thursday, escalate the same morning — don't absorb it silently.

3. Risk register

Risk	Likelihood	Impact	Mitigation
Constraint engine overruns Day 4	Medium	High — blocks Days 5–7	Test-first, pure function, no I/O. FE builds the calendar against MSW mocks and is unblocked regardless. Hard escalation at Thu 12:00.
Atomic class-record save is fiddly (Day 5 PM)	Medium	High	Single Prisma <code>\$transaction</code> , integration test written before the happy path.
Occurrence materialisation edge cases (DST, end dates, mid-week edits)	Medium	Medium	School timezone is fixed (<code>Asia/Kolkata</code> , no DST). Store <code>timestamptz</code> , render in school tz.
Supabase Storage + signed URL setup friction (Day 6)	Low	Medium	Spike it on Day 1 as part of setup; 30 min, not on the critical path.
Design fidelity to the boards slips under time pressure	High	Low–Medium	Tokens + shared components on Day 1 make fidelity the default. Accept minor deviation over missed function.
7 consecutive days including a weekend	—	—	Confirm the calendar before Day 1 (see banner).

4. Cut list — in the order I would cut

If we're behind on Day 6, cut from the top. Each is a deliberate, reversible reduction, not silent scope loss:

1. **PDF weekly report** — a "coming soon" toast. Half a day back.
2. **Level-completion carry-forward UI** — keep the API and the preview; admin picks carry-forward next sprint. Half a day.
3. **Moments video support** — photos only for launch. Quarter day.
4. **Web push infrastructure** (already flagged) — in-app notification centre only. Half a day.
5. **Notification centre UI** — badge counts only, full list next sprint. Quarter day.
6. **Development area admin editing** — ship the seeded catalogue read-only. Quarter day.

Never cut: the atomic save, parent data scoping, the forced-password-change gate, or the empty/loading/error/offline states. The first two are correctness, the third is security, and the fourth is an explicit locked requirement (Flow 04).

5. Solo-developer variant

One engineer instead of two: **12 working days**, same order. Merge each day's BE and FE tracks and run them consecutively — Days 1–2 become Days 1–3, Day 4 stays whole (engine morning, calendar afternoon, +1 day for the UI), Days 6–7 split into four. Do not compress by skipping the Day 1 token/component work; it is what makes Days 5–7 fast.



DEFERRED TO PHASE 2 — AI Integration

Decision D4 (2026-07-27): All AI is deferred. The one-week build ships every flow with a working **manual** path. This document is the complete list of what is *not* built and exactly where each piece plugs in later.

Nothing in the product is blocked by this. Each AI feature is an *input source* for a service that already exists and already works. Phase 2 swaps the input source; it does not rework the flow, the schema, or the UI shell.

1. What is missing, and what ships instead

#	AI feature (Phase 2)	What ships now instead	Flow
AI-1	Voice recording of the class note	<i>No recording step.</i> Teacher goes straight from Attendance to the note screen	07, 16
AI-2	Speech-to-text (English / Telugu / Hindi)	— (nothing to transcribe)	07, 16
AI-3	Split one recording into Overall Class Note + per-student observations	Teacher types the Overall Class Note, then types an observation per present student (each optional except the class note)	07, 16
AI-4	AI-proposed Learning Updates (student + skill + new status)	Teacher picks student → skill → new status from the Learning Map picker inside the Optional Updates step	07, 14, 16
AI-5	AI-proposed Development Observations (student + area + text)	Teacher picks student → development area → types the observation in the Optional Updates step	07, 13, 16
AI-6	AI "Processing your voice note..." progress screen	Not rendered. Stepper goes Attendance → Class Note → Optional Updates → Final Review	07
AI-7	Natural-language class scheduling ("Schedule Aarav for Maths with Priya twice a week...")	Structured Add Class form: student(s), subject, level, teacher, frequency, duration, time preference, start date	06
AI-8	"We understood your request" interpretation-review screen	Not rendered — the form is the structured request, already reviewable and editable	06
AI-9	Voice/NL rescheduling ("Priya is unavailable Friday morning, move her classes...")	Admin opens the Needs Attention issue → sees affected classes → clicks Find New Slots → engine proposes moves	15
AI-10	AI-written "Week at a glance" narrative	Templated assembly from approved learning/development items (Q4 default — already deterministic and in scope)	02

Deliberately not deferred

These read as "AI" on the boards but are plain deterministic code and **ship this week**:

- **The scheduling constraint engine** — capability filter, availability intersection, exception override, conflict detection, recurring-pattern search, Best Match ranking. This is the actual intelligence in scheduling; the AI only ever parsed the sentence.
- **Needs Attention grouping** — rule-based detection and root-cause grouping.
- **Weekly Update item selection** — deterministic query over approved records.
- **Learning-state projection** — `learning_updates` → `student_skill_progress`.

2. Integration seams (already in the code)

Phase 2 is a DI container change plus UI additions. Nothing below gets rewritten.

2.1 Provider interfaces — defined now, manual implementations wired

```
// apps/api/src/ai/contracts.ts      ← exists in week 1
export interface ITranscriptionProvider {
  transcribe(audioPath: string): Promise<{ transcript: string; language: string; seconds: number }>;
}

export interface IClassNoteExtractor {
  extract(input: ExtractionInput): Promise<ClassRecordDraft>;
}

export interface ISchedulingInterpreter {
  interpret(rawText: string): Promise<SchedulingRequestDraft>;
}
```

Week 1 registers `ManualClassNoteExtractor` (returns the teacher's typed payload unchanged as a `ClassRecordDraft`) and `NoopTranscriptionProvider` / `NoopSchedulingInterpreter` (throw `NotImplementedError`, never called because no route reaches them). Phase 2 registers the real ones in `apps/api/src/container.ts`. **No service, controller or route changes.**

2.2 The draft shape is already the contract

`ClassRecordDraft` — the object the review UI renders and `POST /class-records/:id/save` consumes — is identical whether a human typed it or a model produced it:

```
{
  overallClassNote: string;
  studentObservations: { studentId; observation; isAiGenerated: boolean }[];
  proposedLearningUpdates: { studentId; skillId; newStatus; rationale? }[];
  proposedDevelopmentObservations: { studentId; areaId; observation }[];
}
```

`isAiGenerated` and `was_edited` columns exist in `student_observations` from day one and are written `false` this week. This is the single most important seam: **the review-and-approve UI is built now**, so Phase 2 only changes where the draft's initial values come from.

2.3 Schema fields that exist now and stay empty

Table.column	Week 1 value	Phase 2
<code>class_records.transcript</code>	<code>NULL</code>	STT output
<code>class_records.audio_path</code>	<code>NULL</code>	Storage path
<code>class_records.language_detected</code>	<code>NULL</code>	STT detected language
<code>class_records.ai_model</code>	<code>NULL</code>	Model ID
<code>class_records.ai_cost_micros</code>	<code>NULL</code>	Per-record cost
<code>class_records.status</code>	<code>DRAFT → IN_REVIEW → SAVED</code>	adds <code>TRANSCRIBING</code> , <code>PROCESSING</code> , <code>FAILED</code>
<code>student_observations.is_ai_generated</code>	<code>false</code>	<code>true</code> for model-authored
<code>scheduling_requests.raw_text</code>	the form JSON, stringified	the admin's sentence
<code>ai_usage_log</code> (whole table)	empty	one row per AI call

No migration is needed to turn AI on. That is intentional.

2.4 UI placeholders

- **Class record stepper** — a `RecordSourceStep` slot sits between Attendance and Class Note, rendering nothing this week. Phase 2 mounts the recorder + processing screen there.
- **Add Class wizard step 1** — the structured form is wrapped in a `SchedulingInputPanel` with a `mode` prop (`'form'` | `'natural-language'`), hardcoded to `'form'`. Phase 2 adds the textarea + mic and the interpretation-review screen.
- **Needs Attention → Resolve** — routes to the manual reschedule screen. Phase 2 adds a "Describe the change" entry point alongside it.

Every one of these is marked in source with:

```
// TODO(AI-PHASE-2): <what goes here> – see docs/DEFERRED-AI.md §<n>
```

Run `pnpm grep:ai` (alias for `rg "TODO\ (AI-PHASE-2\)"`) at any time for the live list.

3. Phase 2 re-entry checklist

When you're ready, this is the whole job:

1. Decide the model (\$8.3 of the plan has the cost table: Opus 5 ~\$28/mo, Sonnet 5 ~\$15/mo, Haiku 4.5 ~\$11/mo at ~440 records/month).
2. Choose an STT provider that handles English + Telugu + Hindi code-switching (Whisper ~\$0.006/min, or Deepgram).
3. Answer deferred Q6 (model) and Q7 (languages, max recording length, audio retention period).
4. Implement `WhisperTranscriptionProvider` + `ClaudeClassNoteExtractor` + `ClaudeSchedulingInterpreter` against the existing interfaces.
5. Swap the registrations in `container.ts`.
6. Mount the three UI placeholders.
7. Enforce **BR-15**: absent students are stripped from the roster sent to the model *and* filtered from its output.
8. Turn on `ai_usage_log` reporting and the 80% budget alert.

Estimated Phase 2 effort: 3–4 days, because the review, approval, persistence and distribution machinery is already built and tested.

4. Business rules that still apply this week

Deferring AI does not weaken the governance rules — it makes them trivially true:

- **BR-03** (AI never mutates before confirmation) — nothing is AI-generated, and the confirm step still exists.
- **BR-13** (parents see approved outcomes only) — unchanged and fully enforced.
- **BR-02** (Overall Class Note always preserved) — enforced now; it is a required field.
- **BR-19** (atomic multi-part save) — built and integration-tested this week.

The approval layer is the product's spine. It ships first, on purpose, so that AI later plugs into a system that already knows how to say no.